

1. What is MetaMask?

Answer:

MetaMask is a cryptocurrency wallet and a gateway to blockchain applications. It allows users to manage their Ethereum accounts, store Ether and tokens, and interact with decentralized applications (dApps) directly from a web browser or mobile device.

2. What are the steps to install MetaMask?

Answer:

1. Open the Chrome Web Store (or Firefox/Edge Add-ons).
 2. Search for **MetaMask**.
 3. Click “**Add to Browser**” and confirm installation.
 4. Open the extension and click “**Get Started.**”
 5. Create a new wallet or import an existing one using a secret recovery phrase.
 6. Set a password and secure the recovery phrase safely.
-

3. What is a seed phrase (or recovery phrase)?

Answer:

A seed phrase is a 12 or 24-word secret phrase generated by MetaMask during wallet creation. It is used to recover the wallet and all associated funds if the user forgets the password or changes devices.

4. What is Ether (ETH)?

Answer:

Ether (ETH) is the native cryptocurrency of the Ethereum blockchain. It is used to pay for transaction fees (called **gas**) and computational services on the network.

5. What is gas in Ethereum?

Answer:

Gas represents the amount of computational work required to perform operations (like sending transactions or executing smart contracts) on the Ethereum network.

Each operation requires a certain amount of gas, and users pay for it in Ether.

6. What is the difference between gas price and gas limit?

Answer:

- **Gas Price:** The amount of Ether a user is willing to pay per unit of gas (usually measured in gwei).
 - **Gas Limit:** The maximum number of gas units a user is willing to spend for a transaction.
The total transaction cost = **Gas Used × Gas Price.**
-

7. How can you send Ether using MetaMask?

Answer:

1. Open MetaMask and click “**Send.**”
 2. Enter the recipient's Ethereum address.
 3. Enter the amount of Ether to send.
 4. Adjust gas fee (optional).
 5. Click “**Confirm**” to broadcast the transaction.
-

8. What happens when you confirm a transaction in MetaMask?

Answer:

The transaction is broadcast to the Ethereum network, validated by miners (or validators in PoS), and recorded on the blockchain. Ether is deducted from the sender's account including the gas fee.

9. How do you study Ether spending per transaction?

Answer:

After sending a transaction:

- View the transaction details in MetaMask or on **Etherscan** (a blockchain explorer).
 - Check fields like:
 - **Gas Used**
 - **Gas Price**
 - **Transaction Fee (Gas Used × Gas Price)**
This helps analyze how much Ether was spent in total.
-

10. Why do gas fees vary for different transactions?

Answer:

Gas fees depend on:

- Network congestion (more demand → higher gas price)
 - Complexity of the transaction (simple ETH transfer costs less than executing a smart contract)
 - Gas price manually set by the user.
-

11. What happens if you set a very low gas price?

Answer:

The transaction might remain pending for a long time or may not get included in any block until gas prices drop.

12. Can you reverse or cancel a transaction once it's confirmed?

Answer:

No. Once a transaction is confirmed on the Ethereum blockchain, it is irreversible and permanently recorded.

13. What networks can be added to MetaMask besides Ethereum Mainnet?

Answer:

Users can add and interact with **test networks** (like Sepolia, Goerli) or **custom networks** (like Binance Smart Chain, Polygon, Avalanche, etc.) using RPC settings.

14. Why do we use test networks for experiments?

Answer:

To test transactions and smart contracts without spending real Ether. Test networks use **test Ether**, which has no real value.

15. What are the key components displayed in a MetaMask transaction detail?

Answer:

- Transaction Hash
- Status (Success / Pending / Failed)
- From Address
- To Address
- Gas Used

- Gas Price
- Transaction Fee (Ether spent)

◆ Example Calculation of Ether Spent

If:

- Gas Used = 21,000
- Gas Price = 30 gwei

Then:

Transaction Fee = $21,000 \times 30 \text{ gwei} = 630,000 \text{ gwei} = 0.00063 \text{ ETH}$

16. What is Etherscan and why is it used?

Answer:

Etherscan is a blockchain explorer for Ethereum that allows users to track transactions, wallet balances, gas fees, and smart contract details publicly.

17. What is the benefit of using MetaMask for decentralized applications?

Answer:

MetaMask acts as a bridge between users and dApps, allowing secure authentication, transaction signing, and blockchain interactions directly from the browser.

=====

1. What is MetaMask?

Answer:

MetaMask is a crypto wallet and gateway to blockchain applications. It allows users to store, send, and receive cryptocurrencies (mainly Ether and ERC-20 tokens) and interact with decentralized applications (dApps) directly through a web browser or mobile app.

2. Why do we use MetaMask in blockchain applications?

Answer:

MetaMask is used because it provides a secure and user-friendly interface for managing blockchain accounts, signing transactions, and connecting with Ethereum-based dApps without running a full Ethereum node.

3. How do you install MetaMask?

Answer:

1. Go to the **Chrome Web Store** (or Firefox Add-ons).
 2. Search for **MetaMask** and click **Add to Browser**.
 3. After installation, click on the MetaMask icon in the toolbar.
 4. Click **“Get Started”** to begin setting up the wallet.
-

4. How do you create a new wallet in MetaMask?

Answer:

1. Choose **“Create a New Wallet.”**
 2. Set a strong password.
 3. Securely store the **12-word seed phrase (recovery phrase)**.
 4. Confirm the phrase to complete wallet creation.
 5. Your new Ethereum wallet address will be displayed.
-

5. What is a seed phrase or recovery phrase?

Answer:

It is a 12 or 24-word secret phrase generated by MetaMask during wallet creation. It can be used to recover the wallet and all its funds. If lost, the wallet cannot be recovered.

6. What is a wallet address?

Answer:

A wallet address is a unique alphanumeric identifier used to send and receive cryptocurrency on the blockchain.

Example: `0x742d35Cc6634C0532925a3b844Bc454e4438f44e`

7. What type of wallet is MetaMask – custodial or non-custodial?

Answer:

MetaMask is a **non-custodial wallet**, meaning users have full control of their private keys and seed phrase. MetaMask does not store or access user funds.

8. What is the difference between a private key and a public address?

Answer:

- **Public Address:** Shared with others to receive crypto.
 - **Private Key:** A secret cryptographic key used to sign transactions; it must be kept confidential.
-

9. How can you fund your MetaMask wallet?

Answer:

You can add funds by:

- Buying crypto using MetaMask's built-in "Buy" feature.
- Receiving Ether or tokens from another wallet to your MetaMask address.

- Requesting test Ether from a **faucet** if you're on a test network.
-

10. What networks can be connected with MetaMask?

Answer:

MetaMask supports:

- **Ethereum Mainnet** (real Ether transactions)
 - **Testnets** like Sepolia, Goerli (for testing)
 - **Custom networks** like Polygon, BSC, Avalanche, etc., using RPC settings.
-

11. What are the steps to send cryptocurrency using MetaMask?

Answer:

1. Click "**Send**" in MetaMask.
 2. Enter the recipient's wallet address.
 3. Enter the amount of Ether or tokens to send.
 4. Adjust gas fee (optional).
 5. Click "**Confirm.**"
 6. Wait for the transaction to be mined.
-

12. What is a gas fee?

Answer:

A gas fee is the transaction cost paid in Ether to compensate miners or validators for processing and validating transactions on the Ethereum network.

13. How can you view your transaction details?

Answer:

After sending crypto:

- Click on the transaction in MetaMask.
 - View details such as gas fee, transaction hash, and block confirmation.
 - You can also open it in **Etherscan** to see the complete blockchain record.
-

14. What is the importance of Etherscan?

Answer:

Etherscan is a blockchain explorer for Ethereum. It allows users to track wallet balances, view transaction details, gas prices, and smart contract information.

15. Can you recover your MetaMask wallet if you lose your password?

Answer:

Yes, you can recover your wallet using the **12-word recovery phrase**. Without it, access to the wallet and funds is permanently lost.

16. What happens if you enter an incorrect recipient address while sending crypto?

Answer:

If the address is valid, the transaction will go through and cannot be reversed — the crypto will be lost. Ethereum transactions are **irreversible**.

17. What is the difference between the Ethereum Mainnet and test networks?

Answer:

- **Mainnet:** Uses real Ether with real monetary value.
 - **Testnets:** Use test Ether (no real value) for experimenting and development.
-

18. Why is it important to back up your seed phrase?

Answer:

Because the seed phrase is the **only way** to recover your wallet if your device is lost, reset, or your browser data is cleared.

19. What are some security precautions for using MetaMask?

Answer:

- Never share your seed phrase or private key.
 - Use a strong password.
 - Keep the recovery phrase offline.
 - Use hardware wallets for large funds.
 - Beware of phishing websites.
-

20. What are the practical applications of MetaMask in blockchain?

Answer:

- Sending and receiving cryptocurrencies.
- Interacting with decentralized applications (DeFi, NFTs, etc.).
- Testing and deploying smart contracts.
- Managing multiple blockchain accounts.

◆ Example Practical Observation (for record file)

Parameter	Value / Observation
Wallet Type	MetaMask (Non-custodial)
Network Used	Sepolia Testnet
Wallet Address	0x8A3f...b12D
Ether Balance	2.5 ETH (Test Ether)
Transaction Sent To	0x4C6a...aEf2
Amount Sent	0.1 ETH
Gas Used	21,000 units
Gas Price	25 gwei
Transaction Fee	0.000525 ETH
Transaction Status	Successful (Confirmed on Etherscan)

=====

1. What is a smart contract?

Answer:

A smart contract is a self-executing program that runs on the blockchain. It contains predefined rules written in code and automatically enforces them when certain conditions are met, without the need for intermediaries.

2. Which programming language is commonly used for writing Ethereum smart contracts?

Answer:

Solidity is the most widely used programming language for developing smart contracts on the Ethereum blockchain.

3. What is the purpose of this experiment?

Answer:

To write and deploy a smart contract on a test network that simulates basic banking operations such as depositing money, withdrawing money, and checking the account balance of a customer.

4. What is Remix IDE?

Answer:

Remix IDE is a web-based integrated development environment used for writing, compiling, deploying, and testing Solidity smart contracts on the Ethereum blockchain.

5. What are the main functions implemented in the Bank smart contract?

Answer:

1. `deposit()` – Allows the customer to deposit Ether into their account.
 2. `withdraw(uint amount)` – Allows the customer to withdraw a specified amount if sufficient balance is available.
 3. `getBalance()` – Displays the current balance of the customer.
-

6. What is the data type used to store balance in Solidity?

Answer:

The `uint` (**unsigned integer**) data type is used to store the balance because balance values cannot be negative.

7. How do you deploy a smart contract on a test network?

Answer:

1. Write the smart contract in Remix IDE.
 2. Compile it using the Solidity compiler.
 3. Connect Remix to MetaMask.
 4. Select a test network (e.g., **Sepolia** or **Goerli**).
 5. Deploy the contract using test Ether.
 6. Interact with the functions through the Remix interface.
-

8. What is a test network and why do we use it?

Answer:

A **test network** (testnet) is a blockchain network used for testing smart contracts and applications. It uses **test Ether** with no real-world value, allowing developers to safely experiment before deploying on the mainnet.

9. What is the purpose of the `msg.sender` keyword in Solidity?

Answer:

`msg.sender` represents the Ethereum address of the account that initiated the function call or transaction. It is used to identify the sender in smart contract functions.

10. What is the purpose of the `msg.value` keyword?

Answer:

`msg.value` represents the amount of Ether (in wei) sent along with a transaction. It is commonly used in payable functions to record deposits.

11. What does the `payable` keyword mean in Solidity?

Answer:

The `payable` keyword allows a function to receive Ether. Without marking a function as payable, it cannot accept Ether transfers.

12. What is the use of the `require()` statement in Solidity?

Answer:

`require()` is used for validation checks. If the condition inside `require()` fails, the transaction is reverted and the remaining gas is refunded.

Example:

```
require(balance[msg.sender] >= amount, "Insufficient balance");
```

13. Write the basic structure of a Solidity smart contract.

Answer:

```
pragma solidity ^0.8.0;

contract Bank {
    mapping(address => uint) public balance;

    function deposit() public payable {
        balance[msg.sender] += msg.value;
    }

    function withdraw(uint amount) public {
        require(balance[msg.sender] >= amount, "Insufficient
balance");
        payable(msg.sender).transfer(amount);
        balance[msg.sender] -= amount;
    }

    function getBalance() public view returns (uint) {
        return balance[msg.sender];
    }
}
```

14. What is the purpose of the **mapping** data structure in this contract?

Answer:

`mapping(address => uint)` creates a key-value pair where each user's Ethereum address maps to their corresponding account balance.

15. Why is the **withdraw** function marked as non-payable?

Answer:

Because `withdraw()` does not receive Ether; it only sends Ether back to the user. Functions that **send** Ether don't need to be payable.

16. What happens if a user tries to withdraw more Ether than they have deposited?

Answer:

The `require()` condition will fail, and the transaction will revert with the message "Insufficient balance". No Ether will be transferred.

17. How can you view transaction details after executing contract functions?

Answer:

You can check transaction details (gas used, block number, transaction hash, etc.) in Remix IDE logs or on **Etherscan** for the respective test network.

18. What is the smallest unit of Ether in Solidity?

Answer:

The smallest unit of Ether is **wei**.
1 Ether = 10^{18} wei.

19. What is gas in Ethereum transactions?

Answer:

Gas represents the computational cost required to execute operations or functions on the Ethereum network. Every smart contract execution consumes gas, which is paid in Ether.

20. What is the significance of using test Ether instead of real Ether during experiments?

Answer:

Test Ether has **no monetary value** and is used for development and testing. This prevents financial loss while learning or debugging smart contracts.

21. What happens when a transaction fails in Solidity?

Answer:

When a transaction fails (e.g., due to failed `require()` conditions), all state changes are reverted, and the remaining gas is refunded to the sender.

22. What version of Solidity was used in your experiment?

Answer:

The experiment used **Solidity version 0.8.0 or higher**, as it includes built-in overflow and underflow checks.



Optional Observation Table (for your lab record)

Operation	Function Name	Input	Output / Result	Remarks
Deposit Money	<code>deposit()</code>	1 Ether	Balance increased by 1 ETH	Successful
Withdraw Money	<code>withdraw(0.5)</code>	0.5 Ether	Balance decreased by 0.5 ETH	Successful

Show Balance `getBalance()` – Displays remaining balance Successfu
|

=====

1. What is Solidity?

Answer:

Solidity is an object-oriented, high-level programming language used for writing smart contracts that run on the Ethereum blockchain.

2. What is the purpose of this experiment?

Answer:

The purpose is to create a smart contract in Solidity that stores and manages student data using **structures** and **arrays**, and also implements a **fallback function** to handle unexpected Ether transfers.

3. What is a structure (struct) in Solidity?

Answer:

A **structure (struct)** is a user-defined data type that groups related variables under one name. It helps organize complex data like student records (name, roll number, marks, etc.).

Example:

```
struct Student {  
    uint roll;  
    string name;  
    uint marks;  
}
```

4. What is an array in Solidity?

Answer:

An **array** is a data structure used to store multiple elements of the same type. It can be **fixed-size** or **dynamic**.

Example:

```
Student[] public students;
```

5. What is a fallback function in Solidity?

Answer:

A **fallback function** is a special function that is executed when a contract receives Ether without any data, or when a function that doesn't exist is called.

It is defined using:

```
fallback() external payable { }
```

6. What is the purpose of the fallback function in this experiment?

Answer:

To handle unexpected Ether transfers sent to the contract and prevent loss of funds. It ensures the contract can safely receive Ether even when no specific function is called.

7. Write a simple Solidity program to store student data using structures and arrays.

Answer:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract StudentData {
    struct Student {
        uint roll;
        string name;
        uint marks;
    }

    Student[] public students; // Dynamic array of Student structs
```

```
function addStudent(uint _roll, string memory _name, uint _marks)
public {
    students.push(Student(_roll, _name, _marks));
}

function getStudent(uint index) public view returns (uint, string
memory, uint) {
    Student memory s = students[index];
    return (s.roll, s.name, s.marks);
}

// Fallback function
fallback() external payable { }

// To check contract balance
function getBalance() public view returns (uint) {
    return address(this).balance;
}
}
```

8. What is the use of the **memory** keyword in Solidity?

Answer:

memory is used to store temporary data for the duration of a function call. It's used for variables that do not need to be saved on the blockchain permanently.

9. What is the difference between **storage** and **memory**?

Answer:

- **storage:** Data is stored permanently on the blockchain.
 - **memory:** Data is temporary and erased after function execution.
-

10. What is the meaning of **public** in Solidity?

Answer:

public makes variables or functions accessible both within and outside the contract. Solidity automatically generates getter functions for public state variables.

11. What is the purpose of the **payable** keyword?

Answer:

The **payable** keyword allows a function or contract to receive Ether. Without it, the contract cannot accept any Ether transfers.

12. What are gas and transaction fees in Ethereum?

Answer:

- **Gas:** The computational cost of executing operations on the Ethereum network.
 - **Transaction Fee:** The cost paid in Ether, calculated as:
 $\text{Gas Used} \times \text{Gas Price}$.
-

13. How do you deploy a contract on a test network?

Answer:

1. Write and compile the contract in Remix IDE.
 2. Connect MetaMask to Remix.
 3. Select a test network (like Sepolia or Goerli).
 4. Deploy the contract using test Ether.
 5. Observe the **gas used** and **transaction fee** after deployment.
-

14. How can you observe gas and transaction fee values?

Answer:

After deploying the contract in Remix IDE, check the **transaction details**. You can also view them on **Etherscan** by entering the transaction hash.

15. What is the purpose of `address(this).balance` in the program?

Answer:

It returns the total Ether balance currently held by the contract.

16. What happens if Ether is sent to the contract without calling any function?

Answer:

The **fallback function** is executed automatically, allowing the contract to safely accept the Ether.

17. What is the use of the `external` visibility specifier?

Answer:

Functions marked as **external** can only be called from outside the contract (not from within other contract functions).

18. What happens if the contract does not have a fallback or receive function but someone sends Ether to it?

Answer:

The transaction will fail, and the Ether will be reverted back to the sender.

19. What is the difference between `fallback()` and `receive()` functions?

Answer:

- **receive()**: Handles plain Ether transfers with no data.
 - **fallback()**: Handles calls with data or to non-existent functions.
If both exist, **receive()** is used for plain transfers; otherwise, **fallback()** is called.
-

20. What are the steps to interact with this smart contract?

Answer:

1. Deploy the contract on a test network using Remix.
 2. Call **addStudent()** to add new students.
 3. Use **getStudent()** to view stored data.
 4. Send Ether to the contract to trigger the fallback function.
 5. Use **getBalance()** to check Ether stored.
 6. Observe gas and transaction fee values in Remix or Etherscan.
-

21. What is the advantage of using structures and arrays in Solidity?

Answer:

They allow organized and efficient storage of complex data, like maintaining a list of student records in one contract.

22. What is the significance of the **pragma solidity ^0.8.0;** line?

Answer:

It specifies the compiler version to use. The caret (^) means “compatible with version 0.8.0 and above, but below 0.9.0”.

23. What happens when a transaction fails in Solidity?

Answer:

All state changes are reverted, and the remaining gas is refunded to the sender.

24. Can we modify data stored in a struct?

Answer:

Yes, we can update struct data by accessing the array element and assigning new values.

25. Why do we use test networks instead of the Ethereum mainnet for deployment?

Answer:

Test networks use **test Ether** with no real-world value, allowing developers to test and debug contracts safely without financial risk.



Example Observation Table (for your lab record)

Action	Function Used	Gas Used	Transaction Fee (ETH)	Remarks
Deploy Contract	—	175,000	0.0035 ETH	Successful
Add Student	<code>addStudent(</code> <code>)</code>	48,000	0.00096 ETH	Record added
Get Student Data	<code>getStudent(</code> <code>)</code>	22,000	0.00044 ETH	Data retrieved
Send Ether	(fallback)	21,000	0.00042 ETH	Fallback triggered
=====				

1. What is Blockchain?

Answer:

Blockchain is a **distributed and decentralized digital ledger** that records transactions across multiple computers securely and transparently. Each record is stored in a block, and all blocks are linked in a chain using cryptography.

2. What are the main characteristics of Blockchain technology?

Answer:

- Decentralization
- Transparency
- Immutability
- Security
- Consensus Mechanisms
- Traceability

3. What are the main types of Blockchains?

Answer:

1. Public Blockchain
2. Private Blockchain
3. Consortium (Federated) Blockchain
4. Hybrid Blockchain

4. What is a Public Blockchain?

Answer:

A **Public Blockchain** is open to everyone. Anyone can join, read, write, and participate in the network.

Example: Bitcoin, Ethereum.

Use Case: Cryptocurrency transactions and decentralized applications (dApps).

5. What is a Private Blockchain?

Answer:

A **Private Blockchain** is a restricted network where only authorized participants can access or validate transactions.

Example: Hyperledger Fabric, Quorum.

Use Case: Enterprise applications, supply chain management, and banking systems.

6. What is a Consortium (Federated) Blockchain?

Answer:

A **Consortium Blockchain** is semi-decentralized and governed by a group of organizations rather than a single entity.

Example: R3 Corda, Energy Web Foundation.

Use Case: Banking consortia, healthcare data sharing, and inter-company collaborations.

7. What is a Hybrid Blockchain?

Answer:

A **Hybrid Blockchain** combines features of both public and private blockchains. It allows controlled access while maintaining transparency where needed.

Example: Dragonchain, XinFin.

Use Case: Government systems, real estate, and business data sharing.

8. What is the main difference between Public and Private Blockchains?

Answer:

Feature	Public Blockchain	Private Blockchain
Access	Open to everyone	Restricted
Speed	Slower (many nodes)	Faster
Security	Highly secure	Controlled by few entities
Examples	Bitcoin, Ethereum	Hyperledger, Quorum

9. What is a Consortium Blockchain mainly used for?

Answer:

It is used in industries where multiple organizations need to collaborate securely, such as **banking, insurance, and supply chain** sectors.

10. What is decentralization in Blockchain?

Answer:

Decentralization means no single authority controls the network. Every node in the network maintains a copy of the blockchain, ensuring transparency and trust.

11. What are some real-time use cases of Blockchain technology?

Answer:

- **Cryptocurrency transactions** (Bitcoin, Ethereum)
 - **Supply Chain Management** (IBM Food Trust)
 - **Healthcare Data Security**
 - **Voting Systems**
 - **Banking and Finance** (Cross-border payments)
 - **Smart Contracts**
 - **NFTs and Digital Ownership**
-

12. How is Blockchain used in the financial sector?

Answer:

Blockchain enables faster, cheaper, and more transparent transactions. It is used for **digital currencies, cross-border payments, and fraud prevention**.

13. How does Blockchain improve supply chain management?

Answer:

By recording each stage of a product's journey on a transparent ledger, Blockchain improves traceability, reduces fraud, and ensures product authenticity.

14. What is a smart contract?

Answer:

A **smart contract** is a self-executing piece of code stored on the blockchain that automatically enforces agreements when predefined conditions are met.

Example: Automatic payment release in Ethereum when goods are delivered.

15. What are the benefits of Blockchain technology?

Answer:

- Increased transparency
 - Enhanced security
 - Reduced operational costs
 - Improved traceability
 - Faster transactions
 - Elimination of intermediaries
-

16. What are the limitations of Blockchain technology?

Answer:

- High energy consumption

- Scalability issues
 - Data privacy concerns
 - Complex implementation
 - Regulatory uncertainty
-

17. Which consensus mechanisms are used in Blockchains?

Answer:

- **Proof of Work (PoW)** – Used by Bitcoin
 - **Proof of Stake (PoS)** – Used by Ethereum 2.0
 - **Proof of Authority (PoA)** – Used by private networks
 - **Practical Byzantine Fault Tolerance (PBFT)** – Used by Hyperledger Fabric
-

18. What is the importance of gas fees in Ethereum?

Answer:

Gas fees are transaction fees paid to validators for performing computations or storing data on the Ethereum blockchain.

19. What is the role of Blockchain in healthcare?

Answer:

Blockchain ensures **secure storage**, **integrity**, and **privacy** of patient records while enabling interoperability between hospitals and clinics.

20. How can governments use Blockchain technology?

Answer:

Governments can use Blockchain for **land registry**, **public record management**, **voting systems**, and **digital identity verification**.

21. What is the difference between Bitcoin and Ethereum?

Answer:

- **Bitcoin** is primarily a digital currency for peer-to-peer transactions.
 - **Ethereum** is a blockchain platform for building decentralized applications and smart contracts.
-

22. What industries are most impacted by Blockchain technology?

Answer:

- Finance
 - Healthcare
 - Supply Chain
 - Real Estate
 - Government
 - Energy
 - Education
-

23. Why is immutability important in Blockchain?

Answer:

Immutability ensures that once data is recorded on the blockchain, it **cannot be altered or deleted**, providing data integrity and security.

24. What tools are used to analyze Blockchain data?

Answer:

- **Etherscan** (Ethereum Explorer)
 - **Blockchain.com Explorer**
 - **IBM Blockchain Explorer**
 - **Hyperledger Explorer**
-

25. What are some future trends of Blockchain technology?

Answer:

- Integration with Artificial Intelligence (AI)
 - Central Bank Digital Currencies (CBDCs)
 - Tokenization of assets
 - Decentralized Finance (DeFi)
 - Interoperable blockchains
-



Example Summary Table (for your report)

Type of Blockchain	Control Type	Example	Use Case
Public	Decentralized	Bitcoin, Ethereum	Cryptocurrency, dApps
Private	Centralized	Hyperledger Fabric	Banking, Enterprises

Consortium	Semi-Decentralized	R3 Corda	Supply chain, Healthcare
Hybrid	Mixed	Dragonchain	Government, Business networks

Conclusion

Blockchain technology exists in multiple forms—**public, private, consortium, and hybrid**—each suited to different business needs. Its real-world applications are transforming industries by increasing **trust, efficiency, and transparency**.